Target: https://localhost:5000 | **Date:** 2026-07-07 12:14:49 | **Status:** Completed

5	14	10
Critical	High	Medium

3
Low

□ Discovered Vulnerabilities

HIGH Missing Header: Strict-Transport-Security

Affected URL: `https://localhost:5000/`

Vulnerable Parameter:

Payload Used:

Description: Prevents protocol downgrade attacks and cookie hijacking (HSTS).

□□ **Remediation:** Configure your web server to include the 'Strict-Transport-Security' header in all HTTP responses.

HIGH Missing Header: Content-Security-Policy

Affected URL: `https://localhost:5000/`

Vulnerable Parameter:

Payload Used:

Description: Prevents Cross-Site Scripting (XSS) and data injection attacks (CSP).

□□ **Remediation:** Configure your web server to include the 'Content-Security-Policy' header in all HTTP responses.

MEDIUM Missing Header: X-Frame-Options

Affected URL: `https://localhost:5000/`

Vulnerable Parameter:

Payload Used:

Description: Prevents Clickjacking attacks by disabling page embedding in iframes.

□□ **Remediation:** Configure your web server to include the 'X-Frame-Options' header in all HTTP responses.

MEDIUM Missing Header: X-Content-Type-Options

Affected URL: `https://localhost:5000/`

Vulnerable Parameter:

Payload Used:

Description: Prevents the browser from MIME-sniffing the content type, reducing drive-by download risks.

□□ **Remediation:** Configure your web server to include the 'X-Content-Type-Options' header in all HTTP responses.

LOW Missing Header: Referrer-Policy

Affected URL: `https://localhost:5000/`

Vulnerable Parameter:

Payload Used:

Description: Controls how much referrer information is sent when navigating away from the page.

□□ **Remediation:** Configure your web server to include the 'Referrer-Policy' header in all HTTP responses.

LOW Information Disclosure: Verbose Server Header

Affected URL: `https://localhost:5000/`

Vulnerable Parameter:

Payload Used:

Description: The server reveals its exact software version: 'Werkzeug/3.1.8 Python/3.14.6'. Attackers use this to find specific CVEs for your software.

□□ **Remediation:** Configure your web server to hide the 'Server' header or remove the version number (e.g., just say 'Apache' instead of 'Apache/2.4.41').

HIGH Cookie Missing 'Secure' Flag

Affected URL: `https://localhost:5000/login`

Vulnerable Parameter: `session_id`

Payload Used: `N/A`

Description: The cookie 'session_id' is missing the 'Secure' flag, allowing it to be transmitted over unencrypted HTTP connections where it can be intercepted.

□□ **Remediation:** Add the 'Secure' flag to ensure the cookie is only transmitted over HTTPS connections.

HIGH Cookie Missing 'HttpOnly' Flag

Affected URL: `https://localhost:5000/login`

Vulnerable Parameter: `session_id`

Payload Used: `N/A`

Description: The cookie 'session_id' is missing the 'HttpOnly' flag, allowing JavaScript to access it. This enables session theft via XSS attacks.

□□ **Remediation:** Add the 'HttpOnly' flag to prevent client-side scripts from accessing the cookie.

HIGH Cookie Missing 'SameSite' Attribute

Affected URL: `https://localhost:5000/login`

Vulnerable Parameter: `session_id`

Payload Used: `N/A`

Description: The cookie 'session_id' is missing the 'SameSite' attribute or is set to 'None', making it vulnerable to Cross-Site Request Forgery (CSRF) attacks.

□□ **Remediation:** Set the 'SameSite' attribute to 'Strict' or 'Lax' to prevent the cookie from being sent with cross-site requests.

HIGH Cookie Missing 'Secure' Flag

Affected URL: `https://localhost:5000/login`

Vulnerable Parameter: `session`

Payload Used: `N/A`

Description: The cookie 'session' is missing the 'Secure' flag, allowing it to be transmitted over unencrypted HTTP connections where it can be intercepted.

□□ **Remediation:** Add the 'Secure' flag to ensure the cookie is only transmitted over HTTPS connections.

HIGH Cookie Missing 'SameSite' Attribute

Affected URL: `https://localhost:5000/login`

Vulnerable Parameter: `session`

Payload Used: `N/A`

Description: The cookie 'session' is missing the 'SameSite' attribute or is set to 'None', making it vulnerable to Cross-Site Request Forgery (CSRF) attacks.

□□ **Remediation:** Set the 'SameSite' attribute to 'Strict' or 'Lax' to prevent the cookie from being sent with cross-site requests.

HIGH Reflected Cross-Site Scripting (XSS)

Affected URL: `https://localhost:5000/user?id=1`

Vulnerable Parameter: `id`

Payload Used: `">`

Description: The parameter 'id' reflects user input without proper encoding, allowing script execution.

□□ **Remediation:** Use Context-Aware Output Encoding. Convert special characters to HTML entities before rendering.

HIGH Reflected Cross-Site Scripting (XSS)

Affected URL: `https://localhost:5000/view-doc?filename=welcome.txt`

Vulnerable Parameter: `filename`

Payload Used:

Description: The parameter 'filename' reflects user input without proper encoding, allowing script execution.

□□ **Remediation:** Use Context-Aware Output Encoding. Convert special characters to HTML entities before rendering.

HIGH Reflected Cross-Site Scripting (XSS)

Affected URL: `https://localhost:5000/fetch-data?url=https://example.com`

Vulnerable Parameter: `url`

Payload Used:

Description: The parameter 'url' reflects user input without proper encoding, allowing script execution.

□□ **Remediation:** Use Context-Aware Output Encoding. Convert special characters to HTML entities before rendering.

CRITICAL Error-Based SQL Injection

Affected URL: `https://localhost:5000/user?id=1`

Vulnerable Parameter: `id`

Payload Used: `'`

Description: The application throws a database error when a single quote is injected, indicating unsanitized SQL queries.

□□ **Remediation:** Use parameterized queries or prepared statements instead of string concatenation/interpolation.

HIGH Exposed Sensitive Path: /.env

Affected URL: `https://localhost:5000/.env`

Vulnerable Parameter: `N/A`

Payload Used: `HTTP 200`

Description: The path '/.env' returned a 200 status code. It is Publicly Accessible.

□□ **Remediation:** Remove the file/directory from the web root or configure the web server to deny access to it.

HIGH Exposed Sensitive Path: /admin

Affected URL: `https://localhost:5000/admin`

Vulnerable Parameter: `N/A`

Payload Used: `HTTP 200`

Description: The path '/admin' returned a 200 status code. It is Publicly Accessible.

□□ **Remediation:** Remove the file/directory from the web root or configure the web server to deny access to it.

MEDIUM Missing CSRF Token on POST Form

Affected URL: `https://localhost:5000/`

Vulnerable Parameter: `/parse-xml`

Payload Used: `N/A`

Description: The form submits data via POST but lacks an anti-CSRF token, making it vulnerable to Cross-Site Request Forgery attacks where malicious sites can submit forms on behalf of authenticated users.

□□ **Remediation:** Implement anti-CSRF tokens in all state-changing forms. Generate a unique token per session, include it as a hidden field in forms, and validate it on the server side before processing the request.

MEDIUM Missing CSRF Token on POST Form

Affected URL: `https://localhost:5000/transfer`

Vulnerable Parameter: `/transfer`

Payload Used: `N/A`

Description: The form submits data via POST but lacks an anti-CSRF token, making it vulnerable to Cross-Site Request Forgery attacks where malicious sites can submit forms on behalf of authenticated users.

□□ **Remediation:** Implement anti-CSRF tokens in all state-changing forms. Generate a unique token per session, include it as a hidden field in forms, and validate it on the server side before processing the request.

MEDIUM Missing CSRF Token on POST Form

Affected URL: `https://localhost:5000/ping`

Vulnerable Parameter: `/ping`

Payload Used: `N/A`

Description: The form submits data via POST but lacks an anti-CSRF token, making it vulnerable to Cross-Site Request Forgery attacks where malicious sites can submit forms on behalf of authenticated users.

□□ **Remediation:** Implement anti-CSRF tokens in all state-changing forms. Generate a unique token per session, include it as a hidden field in forms, and validate it on the server side before processing the request.

MEDIUM Missing CSRF Token on POST Form

Affected URL: `https://localhost:5000/login`

Vulnerable Parameter: `/login`

Payload Used: `N/A`

Description: The form submits data via POST but lacks an anti-CSRF token, making it vulnerable to Cross-Site Request Forgery attacks where malicious sites can submit forms on behalf of authenticated users.

□□ **Remediation:** Implement anti-CSRF tokens in all state-changing forms. Generate a unique token per session, include it as a hidden field in forms, and validate it on the server side before processing the request.

MEDIUM Self-Signed SSL Certificate Detected

Affected URL: `https://localhost:5000/`

Vulnerable Parameter: `N/A`

Payload Used: `N/A`

Description: The certificate for localhost is self-signed (issuer matches subject:), meaning it is not trusted by a recognized Certificate Authority.

□□ **Remediation:** Use a certificate issued by a trusted Certificate Authority (e.g. Let's Encrypt) for any publicly accessible service.

LOW SSL Cipher Suite Information

Affected URL: `https://localhost:5000/`

Vulnerable Parameter: `N/A`

Payload Used: `N/A`

Description: Connection negotiated using TLSv1.3 with cipher TLS_AES_256_GCM_SHA384 (256-bit).

□□ **Remediation:** Verify the negotiated protocol/cipher meet current best practices (TLS 1.2+ with strong AEAD ciphers).

CRITICAL OS Command Injection

Affected URL: `https://localhost:5000/ping`

Vulnerable Parameter: `target`

Payload Used: `127.0.0.1 & echo VULN_CMD_INJECTION_SUCCESS`

Description: The application passes user input directly to the OS shell, allowing arbitrary command execution.

□□ **Remediation:** Never pass user input directly to os.system() or subprocess with shell=True. Use subprocess arrays without shell=True, or strict allow-lists for expected input.

HIGH Insecure Direct Object Reference (IDOR)

Affected URL: `https://localhost:5000/user?id=1`

Vulnerable Parameter: `id`

Payload Used: `id=2`

Description: The application allows authenticated users to access data belonging to other users by simply modifying the ID in the URL.

□□ **Remediation:** Implement strict access controls. Never rely on client-side input for object references. Verify that the logged-in user has explicit permission to access the requested resource.

CRITICAL Path Traversal / Local File Inclusion (LFI)

Affected URL: `https://localhost:5000/view-doc?filename=welcome.txt`

Vulnerable Parameter: `filename`

Payload Used: `..\..\..\..\Windows\win.ini`

Description: The application allows reading arbitrary files from the server's filesystem by manipulating file path parameters.

□□ **Remediation:** Never pass user input directly to file system APIs. Use an allow-list of permitted files, or implement strict input validation to reject path traversal characters like '../'.

CRITICAL Server-Side Request Forgery (SSRF)

Affected URL: `https://localhost:5000/fetch-data?url=https://example.com`

Vulnerable Parameter: `url`

Payload Used: `https://localhost:5000/`

Description: The application fetches user-supplied URLs without validation, allowing attackers to force the server to make requests to internal networks, cloud metadata services, or bypass firewalls.

□□ **Remediation:** Implement a strict allow-list of permitted domains. Block requests to internal IP ranges (127.0.0.1, 10.x.x.x, 192.168.x.x, 169.254.x.x) and disable unnecessary URL schemes (like file:// or gopher://).

HIGH CORS Misconfiguration - Arbitrary Origin Reflection

Affected URL: `https://localhost:5000/api/user-data`

Vulnerable Parameter: `Origin Header`

Payload Used: `Origin: https://evil.com`

Description: The server reflects arbitrary Origin headers with credentials allowed, enabling attackers to make authenticated cross-origin requests from malicious websites and steal user data.

□□ **Remediation:** Implement a strict allow-list of permitted origins. Never reflect the Origin header dynamically. Use specific domain names instead of wildcards when credentials are involved.

CRITICAL XML External Entity (XXE) Injection

Affected URL: `https://localhost:5000/parse-xml`

Vulnerable Parameter: `XML Body`

Payload Used: `]> &xxe;`

Description: The application processes XML input with external entity resolution enabled, allowing attackers to read local files, perform SSRF attacks, or cause denial of service.

□□ **Remediation:** Disable external entity processing in XML parsers. Use defusedxml library instead of standard XML parsers. Never process untrusted XML input with entity resolution enabled.

MEDIUM Open Redirect

Affected URL: `https://localhost:5000/redirect?next=/dashboard`

Vulnerable Parameter: `next`

Payload Used: `https://evil.com`

Description: The application redirects users to arbitrary external URLs without validation, enabling phishing attacks and OAuth bypass vulnerabilities.

□□ **Remediation:** Validate redirect URLs against an allow-list of permitted domains. Never redirect to user-supplied URLs without verification. Use relative paths for internal redirects.

MEDIUM Open Redirect

Affected URL: `https://localhost:5000/redirect`

Vulnerable Parameter: `next`

Payload Used: `https://evil.com`

Description: The application redirects users to arbitrary external URLs without validation, enabling phishing attacks and OAuth bypass vulnerabilities.

□□ **Remediation:** Validate redirect URLs against an allow-list of permitted domains. Never redirect to user-supplied URLs without verification. Use relative paths for internal redirects.

MEDIUM Open Redirect

Affected URL: `https://localhost:5000/redirect`

Vulnerable Parameter: `url`

Payload Used: `https://evil.com`

Description: The application redirects users to arbitrary external URLs without validation, enabling phishing attacks and OAuth bypass vulnerabilities.

□□ **Remediation:** Validate redirect URLs against an allow-list of permitted domains. Never redirect to user-supplied URLs without verification. Use relative paths for internal redirects.